

# Project Setup & Initial LLM Integration Report: Inktellect

## 1. Project Overview

**Project Name:** Inktellect: An AI Learning Buddy

**Developer:** Mary Rose La Torre

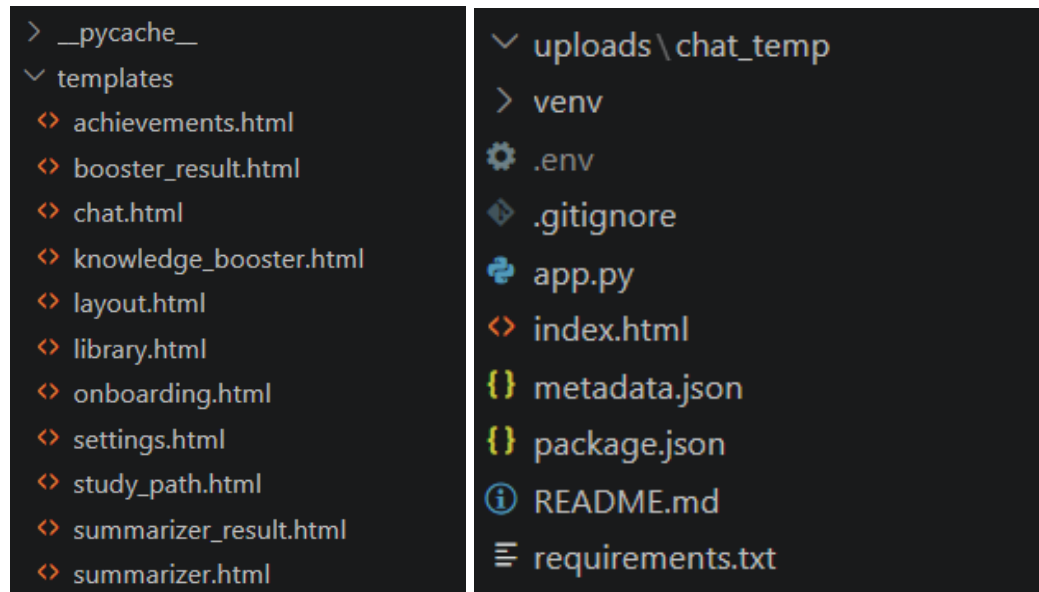
**Date:** May 2026

**Executive Summary:** Inktellect is a personalized AI-powered learning companion designed to help students interact with study materials using Retrieval-Augmented Generation (RAG). This report details the foundational setup, directory architecture, and the integration of the Groq LLM engine.

## 2. Technical Stack & Justification

| Component         | Technology            | Justification                                                                         |
|-------------------|-----------------------|---------------------------------------------------------------------------------------|
| Backend Framework | Flask (Python)        | Chosen for granular control over UI/UX and session-based state management.            |
| LLM Provider      | Groq (Llama 3.3 70B)  | Selected for industry-leading inference speed and low latency for the "Octi" persona. |
| Vector Database   | FAISS                 | Used for local, privacy-first document indexing and similarity search.                |
| Embedding Model   | Sentence-Transformers | Uses all-MiniLM-L6-v2 for lightweight, accurate text-to-vector conversion.            |

## 3. Core Directory Layout



| Directory / File   | Description                                                   |
|--------------------|---------------------------------------------------------------|
| uploads/chat_temp/ | Root for user files and temporary chat-specific attachments.  |
| templates/         | HTML Frontend using Jinja2 (onboarding, chat, library, etc.). |
| app.py             | Main Flask application containing the core RAG logic.         |
| .env               | Secure storage for environment variables (API Keys).          |
| requirements.txt   | Comprehensive list of project dependencies.                   |
| venv/              | Isolated Python virtual environment.                          |

## 4. Setup & Integration Process

### A. Environment & Dependency Management

- **Security:** A `.env` file manages `GROQ_API_KEY` and `FLASK_SECRET_KEY` to prevent credential leaks.
- **Installation:** The environment is built using `pip install -r requirements.txt`, including PyPDF2 and python-docx for parsing.

B. LLM Initialization

The system uses the **Groq SDK** to maintain a "Context-Aware" persona. The LLM adjusts its tone and complexity by reading user data (e.g., Grade Level: Elementary) directly from the Flask session.

5. Initial RAG Implementation

To prevent LLM "hallucinations," a `RAGManager` class was implemented with the following workflow:

| Step | Action          | Process                                                                         |
|------|-----------------|---------------------------------------------------------------------------------|
| 1    | Text Extraction | Files are parsed and cleaned of artifacts (extra spaces/newlines).              |
| 2    | Chunking        | Documents are split into 500-word segments with overlap for context.            |
| 3    | Indexing        | Chunks are embedded and stored in a unique <code>session_id</code> FAISS index. |

6. Challenges & Solutions

| Challenge           | Solution                                                                                |
|---------------------|-----------------------------------------------------------------------------------------|
| Fragmented PDF Text | Created a <code>_clean_extracted_text</code> function using Regex to join broken words. |
| Session Overlap     | Utilized uuid to generate unique session IDs, isolating document contexts.              |
| Latency             | Integrated Groq's LPUs to ensure near-instantaneous responses for the student.          |